Çankaya University

Computer Engineering Department

CENG-408

Methodology Report

Team Members:

Muhammed Yusuf Özcan

Alperen Berke Çetinkaya

Sezer Ataş

Mete Serpil

Content

Methodology

1. Study Scope and Methodological Objective

This study presents an offline, case-specific brain tumor MRI decision-support prototype that transforms segmentation outputs into quantitative, anatomically grounded, and clinically readable artifacts. The system is not designed as an autonomous diagnostic tool. Instead, it acts as a post-segmentation interpretation framework that connects tumor masks to measurable descriptors, approximate neuroanatomic explanations, structured question-answering outputs, and draft clinical reports.

The central methodological principle of the project is grounded generation. In practice, this means that the system should remain anchored to explicit case data while still producing outputs that are readable and useful for clinicians. For that reason, the architecture deliberately separates the deterministic measurement layer from the controlled narrative layer. Quantitative measurements, lesion counts, and structured descriptors are produced deterministically, whereas natural-language fluency is added selectively through a constrained local language model.

2. Overall System Workflow

At a high level, the pipeline can be summarized as follows:

MRI case package → segmentation mask → quantitative tumor metrics → atlas-aware anatomic descriptors → grounded report and QA generation

The workflow begins with the organization of MRI sequences at the case level. After segmentation is available, the mask is converted into clinically relevant volumetric and lesion-centric measurements. These measurements are then enriched with approximate anatomic localization through atlas-based mapping. Finally, the resulting structured case representation is used for report generation and question answering.

Methodologically, this is a hybrid system. The numeric truth layer is deterministic, reproducible, and traceable to structured fields. The explanatory layer is partially generative but constrained by curated descriptors, safety rules, and validation logic.

3. Case Discovery and Data Preparation

3.1 Case-Level Packaging

Each case is stored in a dedicated folder containing multimodal MRI sequences, typically including T1, T1ce, T2, and FLAIR images in NIfTI format. The system scans the case directory, identifies available files, and derives a minimal study-level description of the input package. This stage does not perform medical reasoning; its purpose is to create a stable and reproducible case context for all downstream stages.

3.2 Structured Patient Context

The discovered imaging inputs are transformed into a structured patient_context representation. This object stores identifiers such as patient ID, study ID, study date, modality, and imaging availability summaries. It also acts as the shared case-level substrate for both reporting and question answering. Because both downstream components read from the same context object, the system remains case-locked: outputs are produced from the currently loaded case package rather than from unconstrained general medical knowledge.

4. Segmentation Layer

4.1 Segmentation as a Prerequisite Layer

The segmentation stage follows an nnU-Net-style inference workflow. MRI sequences are prepared in the expected channel format, and segmentation masks are generated as NIfTI outputs. In this project, segmentation is treated as a prerequisite computational layer rather than the main scientific contribution. This design choice is consistent with the literature, where nnU-Net has become a strong baseline for biomedical segmentation and where multimodal glioma MRI segmentation has been extensively studied through the BraTS benchmark ecosystem.

4.2 Methodological Role of Segmentation

The main contribution of this study is therefore not a new segmentation model. Instead, the contribution lies in what happens after segmentation: the conversion of a mask into measurable, explainable, and reportable clinical artifacts. This positioning is important, because it defines the project as an integrative interpretation system rather than as a benchmark-optimizing segmentation paper.

5. Quantitative Tumor Metrics Extraction

5.1 From Mask to Physical Measurements

Once a segmentation mask is available, the system computes quantitative tumor metrics directly from it. This stage is one of the most important parts of the methodology because it converts a voxelwise label image into clinically interpretable measurements. The mask is read together with voxel spacing information so that all volumetric values can be expressed in physical units such as cubic millimeters and cubic centimeters.

5.2 Core Metrics

The extracted metrics include:

- total segmented tumor volume
- maximum segmented lesion diameter
- label-specific component volumes
- lesion list
- merged lesion-group count
- segmented component count

Label-specific burden is computed separately for enhancing tumor, non-enhancing tumor, and edema. These quantities are preserved both as absolute measurements and as proportions of the total segmented burden.

5.3 Lesion Groups Versus Raw Components

An important design decision is the distinction between lesion groups and segmented components. The system does not simply count every connected fragment as an independent lesion. Instead, compatible tumor labels are merged first so that the resulting lesion-group count better reflects biologically meaningful lesion structure. This reduces the risk of overcalling multifocal disease when a single heterogeneous lesion appears fragmented in the segmentation mask. In addition, very small fragments can be filtered using a minimum voxel threshold so that obvious segmentation debris does not dominate the case interpretation.

6. Lesion-Centric Representation

Beyond global case-level summaries, the system preserves a lesion-centric data model. Each meaningful component in the segmentation is represented in a lesion_list with fields such as lesion ID, label type, volume, maximum diameter, centroid, and anatomic descriptor. This design is methodologically important because it avoids collapsing the entire case into a single summary number.

The lesion-centric representation supports clinically meaningful downstream questions, including:

- identification of the largest lesion
- characterization of the dominant component
- assessment of whether the burden is concentrated in one dominant lesion with satellite groups
- localization of specific lesion components

This intermediate representation is also reused in both the reporting and QA stages, improving consistency across system outputs.

7. Atlas-Backed Anatomic Mapping

7.1 Approximate Neuroanatomic Localization

Quantitative measurements alone are not sufficient for clinically useful explanation. For this reason, the pipeline includes an atlas-backed anatomic mapping stage. Lesions are approximately aligned into standard space and sampled against the Harvard-Oxford atlas. This produces descriptors such as hemisphere, lobe, depth, superior-middle-inferior level, and, when supported, atlas-linked regional labels.

7.2 Uncertainty-Aware Localization Language

The project does not treat these localization outputs as exact anatomical truth. Instead, atlas confidence and registration quality are explicitly preserved and then propagated into the final language layer. When localization support is limited, the final wording becomes more cautious and uses terms such as "approximate atlas-supported overlap." This is methodologically significant because uncertainty is modeled as part of the pipeline itself rather than hidden beneath a single definitive region name.

8. Registration Quality Awareness

Atlas mapping is not treated as a binary success-or-failure step. Instead, the system retains registration quality signals, overlap-related indicators, and related mapping quality metadata. These values do not function merely as hidden debug fields; they inform how strongly localization statements may be expressed in the final outputs.

When localization support is relatively strong, the report may describe the dominant component as atlas-backed but approximate. When support is weak, the language is downgraded accordingly. This makes confidence-sensitive reporting part of the explanatory design rather than an afterthought.

9. Structured Descriptor Layer

9.1 Compact Intermediate Representation

The system does not pass the entire raw case context directly into the language model. Instead, it builds a compact report_descriptor containing semantically meaningful intermediate abstractions such as:

- case overview

- segmentation overview
- component profile
- dominant lesion summary
- spatial distribution
- semantic segmentation summary
- narrative hints

9.2 Why the Descriptor Layer Matters

This descriptor layer serves several purposes at once. It reduces prompt size, improves semantic organization, lowers truncation risk, and constrains the language model to grounded intermediate variables rather than loosely structured raw metadata. In practice, this is one of the main reasons the system maintains better numerical discipline than a fully free-form report generator.

This design is also aligned with recent grounded medical report-generation literature, where explainable intermediate representations are used to stabilize downstream natural-language output.

10. Hybrid Report Generation

10.1 Division of Responsibilities

The reporting methodology is intentionally hybrid. The deterministic layer is responsible for report structure, mandatory section headings, quantitative findings, safety-critical wording, and disclaimer boundaries. The language model is then used in a constrained role to improve fluency in selected narrative sections, especially Clinical Context, Findings, and Impression.

10.2 Report Structure

The report is organized under fixed high-level sections:

- Clinical Context
- Findings
- Impression
- Limitations
- Suggested Clinical Correlation
- Disclaimer

Within Findings, the system may further present localization, regional distribution, enhancement pattern, multiplicity, and related automatically derived descriptors. In current versions of the project, heuristic burden-related statements are deliberately softened and presented as secondary automated flags rather than direct radiologic measurements.

10.3 Soft Repair Instead of Hard Rejection

An important refinement in the report pipeline is the move from hard rejection to soft repair. Earlier versions of the system tended to discard generated narrative entirely when it contained unsafe or overly strong wording. In the current design, the report layer first attempts to repair such text while preserving useful narrative content. This produces a more readable report without giving up deterministic control over core measurements and safety boundaries.

11. Controlled Question Answering

11.1 QA Decision Process

The QA engine follows a structured pipeline:

- normalize the incoming question
- classify the question type
- retrieve the relevant structured fields
- generate a deterministic answer when possible
- optionally add constrained narrative support
- attach evidence IDs, confidence, and safety notes

11.2 Conservative Behavior for High-Risk Questions

For quantitative questions, answers are produced directly from structured metrics. For higher-risk questions, especially those involving diagnosis, treatment, prognosis, chemotherapy, surgery, or cure rate, the system applies explicit guardrails. In these situations, the system either refuses to make a definitive claim or explains what is known from the current case package and what additional information would be required for valid clinical decision-making.

This makes the QA layer intentionally stricter than the report layer. Methodologically, that asymmetry is deliberate: reports benefit from broader narrative continuity, whereas question answering must prioritize direct evidence linkage, traceability, and conservative safety behavior.

12. Evidence Tracking and Evaluation

12.1 Evidence-Linked Outputs

A major strength of the methodology lies in evidence tracking. Each QA response stores the answer text, evidence identifiers, confidence, and safety notes. Evidence identifiers can then be resolved back to the exact structured context fields from which the answer was derived. This gives the project an auditable path from output text back to case-level source data.

12.2 Run-Level Artifacts

Each report run is accompanied by a full artifact set, including:

- report draft
- QA outputs
- evidence mappings
- quality summaries
- evaluation logs
- comparison outputs
- run metadata

The run metadata additionally stores prompt policy, model version, question-set provenance, and run fingerprinting information. This transforms the system from a one-off demo into a reproducible workflow.

12.3 Evaluation Scope

The evaluation layer measures report and QA quality through metrics such as:

- section completeness
- QA evidence coverage

- unsupported statement rate
- contradiction detection
- readability
- lexical diversity

These metrics do not prove clinical validity by themselves, but they do provide a structured way to assess consistency, traceability, and output quality at the system level.

13. Methodological Positioning in the Literature

Within the existing literature, the project should be positioned as an integrative contribution rather than a foundational algorithmic one. It does not introduce a new segmentation architecture beyond established approaches such as nnU-Net, nor does it attempt to outperform benchmark-oriented segmentation studies such as BraTS.

Its contribution lies instead in the integration of:

- segmentation-derived quantitative metrics
- lesion-centric representation
- atlas-aware explanation
- structured reporting
- grounded question answering
- safety controls
- reproducible artifact generation

Compared with structured reporting frameworks such as BT-RADS, the system is less clinically mature but more automated at the case-processing level. Compared with LLM-first report-generation approaches, it is more conservative and more tightly grounded in deterministic case data. Compared with segmentation-only studies, it contributes less at the model-innovation level but more at the level of explainability and downstream usability.

14. Limitations

Several methodological limitations remain. First, the system does not directly measure mass effect or midline shift through dedicated image analysis; these remain heuristic secondary flags rather than validated imaging biomarkers. Second, atlas localization is approximate and may become unstable when registration quality is poor. Third, the report generator still retains some template structure, especially around quantitative support lines and mandatory safety statements. Fourth, all downstream explanation quality depends on segmentation quality. If the mask is incorrect, the explanation may remain internally consistent while still being clinically misleading.

For these reasons, the system should be presented as a research or educational decision-support prototype rather than as a clinical diagnostic system.

15. Conclusion of the Methodological Design

In summary, the methodology of this project is built around a hybrid, safety-aware, segmentation-driven interpretation pipeline for brain tumor MRI. Its key contribution is the integration of deterministic quantitative analysis, approximate atlas-based localization, controlled narrative generation, evidence-linked QA, and reproducible evaluation into a single offline workflow.

This makes the project well suited to a final-year engineering context: not because it introduces a completely new algorithmic paradigm, but because it meaningfully combines multiple strands of current literature into a practical and explainable prototype.

16. Lung Nodule Segmentation Methodology

This section describes the methodology used for automated lung nodule segmentation on CT images, complementing the brain tumor segmentation pipeline described in Sections 4–5. The evaluation was performed on the Medical Segmentation Decathlon Task06_Lung dataset, consisting of 63 chest CT scans with annotated lung nodule masks.

17. Dataset

Source: Medical Segmentation Decathlon, Task06_Lung (Simpson et al., 2019). The dataset consists of 63 three-dimensional chest CT volumes with binary voxel-level annotation masks (label=1: nodule, label=0: background). Image dimensions follow a 512 × 512 × N slice layout with N varying per case. All images are provided in NIfTI format.

18. Model

The segmentation model is based on the nnU-Net v1 framework (Isensee et al., 2021) using the 3D Full Resolution (3d_fullres) configuration. The model was trained with 5-fold cross-validation over 1000 epochs per fold using the officially released pretrained weights for Task06_Lung. No retraining was performed; the model was used solely for inference.

The architecture is a self-configuring U-Net with automated preprocessing, patch size selection, and postprocessing. Training employed a compound loss combining Dice Loss and Cross-Entropy, optimized via SGD with Nesterov momentum and polynomial learning rate decay.

19. Inference Pipeline

19.1 Input Preparation

CT images were converted to NIfTI format (.nii.gz) and renamed according to nnU-Net's required naming convention: {case_id}_0000.nii.gz (e.g., lung_001_0000.nii.gz).

19.2 Running Inference

Inference was performed using all five trained folds in ensemble mode. The following command was used to run prediction:

```
nnUNet_predict.exe -i <input_dir> -o <output_dir> -t Task006_Lung -m 3d_fullres
```

All five folds (fold_0 through fold_4) were used; predictions were averaged before thresholding. Test-time augmentation (mirroring) was enabled by default. Output masks were written in NIfTI format (.nii.gz).

19.3 Postprocessing

nnU-Net applies its own learned postprocessing strategy automatically based on the training configuration. No additional manual postprocessing was applied to the predictions.

20. Evaluation

20.1 Metrics

Performance was evaluated at the voxel level using the following metrics:

Metric	Formula
Dice Similarity Coefficient	$2 \cdot TP / (2 \cdot TP + FP + FN)$
Precision	$TP / (TP + FP)$
Recall (Sensitivity)	$TP / (TP + FN)$

TP, FP, and FN denote true positives, false positives, and false negatives at the voxel level with binary label=1 (nodule). Summary statistics were computed using macro-averaging: per-case metrics were calculated independently and then averaged across all cases.

20.2 Ground Truth Matching

Predictions were matched to ground truth masks by exact filename (e.g., lung_001.nii.gz ↔ lung_001.nii.gz). Shape consistency was verified for each case prior to metric computation.

20.3 Results

Evaluation was performed over all 63 cases. One case (lung_096) was identified as an outlier due to a resampling artifact: the model correctly localised the nodule in the right z-range but with an approximately 50-voxel offset along the y-axis, resulting in zero spatial overlap despite both the prediction and ground truth containing nodule voxels. This case is reported separately.

Full 63-case evaluation:

Metric	Mean ± Std	Min	Max
Dice	0.8888 ± 0.1213	0.0000	0.9747
Precision	0.8968 ± 0.1302	0.0000	0.9776
Recall	0.8841 ± 0.1211	0.0000	0.9826

62-case evaluation (lung_096 outlier excluded):

Metric	Mean ± Std	Min	Max
Dice	0.9031 ± 0.0449	0.6936	0.9747
Precision	0.9112 ± 0.0451	0.5746	0.9776
Recall	0.8984 ± 0.0467	0.6760	0.9826

Notable cases:

Case	Dice	Note
lung_014	0.9747	Best performing case
lung_073	0.6936	Lowest (excluding outlier)
lung_096	0.0000	Resampling artifact outlier

21. Reproducibility

Metric computation was independently verified using two separate evaluation scripts: (1) a custom Python script using nibabel with macro-averaged metrics saved to JSON, and (2) an independent reproduction using the same ground truth source and metric definitions. Both approaches yielded identical results (Dice: 0.8888 ± 0.1213), confirming the correctness of the reported metrics.

22. Unity Methodology

The system is built on Unity 6 (version 6000.3.7f1) with the Universal Render Pipeline (URP) 17.3.0, targeting the Meta Quest 3 headset (Qualcomm Snapdragon XR2 Gen 2, Adreno 740 GPU). VR integration is provided by the Meta XR SDK All-in-One package (v85.0.0) running on the OpenXR 1.16.1 backend. Volume rendering is handled by the UnityVolumeRendering open-source library, integrated as a Git-based Unity package. Compute-intensive CPU tasks such as mesh generation leverage the Unity Jobs System and Burst Compiler for parallelism. The input data format is NIfTI-1 (.nii and .nii.gz), the standard format for neuroimaging and medical volumetric data.

Component	Implementation
Game Engine	Unity 6000.3.7f1 (Unity 6 LTS)
Render Pipeline	Universal Render Pipeline (URP) 17.3.0
VR SDK	Meta XR SDK All-in-One 85.0.0 (OpenXR 1.16.1)
Volume Rendering	UnityVolumeRendering (mlavik1, Git package)
Target Hardware	Meta Quest 3 (Snapdragon XR2 Gen 2, Adreno 740)
Data Format	NIfTI-1 (.nii, .nii.gz)
Parallelism	Unity Jobs System + Burst Compiler
Automation	Windows Batch Script + Unity Editor C# API

23. Data Ingestion Pipeline

23.1 NIfTI File Reading

Two NIfTI reading pathways are used in the system. For the primary CT volume rendering pipeline, the UnityVolumeRendering library's built-in importer (IImageFileImporter with ImageFileFormat.NIFTI) reads the NIfTI file and produces a VolumeDataset containing a GPU-resident 3D texture. For the segmentation mesh pipeline, a custom reader (NIfTIReader.cs) was implemented to parse the 348-byte NIfTI-1 header and extract volume dimensions (dimX, dimY, dimZ), voxel spacing (pixX, pixY, pixZ), data type, bit depth, and slope/intercept scaling parameters. The reader supports .nii.gz files via System.IO.Compression.GZipStream decompression and handles

eight NIfTI data types: unsigned byte, signed/unsigned 16-bit and 32-bit integers, float32, float64, and signed byte. The output is an integer label array where each voxel holds its segmentation class ID.

23.2 Automation Pipeline

The entire import-to-scene workflow is driven by a Windows batch script (Run_Unity_Medical_Auto.bat) that launches Unity with the -executeMethod flag, invoking AutoMedicalImport.RunImportPipeline(). All configuration parameters, CT file path, scene path, view distance, viewport fill fraction, Meta XR feature flags, and the volume rendering sampling rate, are defined as environment variables in the batch file and passed as command-line arguments. The C# pipeline executes the following steps:

1. Opens the target scene (or copies it from a template).
2. Cleans up default scene objects (Main Camera, Global Volume, leftover cross-section planes).
3. Deletes any pre-existing VolumeRenderedObject and SingleMeshBuilder instances.
4. Imports the CT NIfTI file through UnityVolumeRendering and spawns the volume object.
5. Attaches transfer function components (CTLungTransferFunction with runtime toggle).
6. Attaches performance optimization components (QuestPerformanceSetup, QuestVolumeOptimizer).
7. Spawns an interactive cross-section slicing plane on the volume.
8. Installs Meta XR Building Blocks asynchronously (Camera Rig, Passthrough, Hand Tracking, Interaction, Hand Grab) with automatic singleton detection to prevent duplicate installations.
9. Auto-frames the volume to the VR camera using configurable view distance and viewport fill parameters.
10. Saves the scene and all assets.

Asynchronous Building Block installations are polled via EditorApplication.update with a 120-second timeout, and the pipeline gracefully handles installation cancellations or failures without aborting

24. Volume Rendering

24.1 Ray Marching Algorithm

The core rendering technique is GPU-based direct volume rendering (DVR) using front-to-back ray marching. For each fragment, a ray is cast from the camera through the volume's axis-aligned bounding box (AABB). The ray's entry and exit points are computed via AABB intersection, and the ray is sampled at uniform intervals determined by the step count. At each sample point, the voxel density is read from a 3D texture via trilinear interpolation, mapped through a 1D transfer function to obtain RGBA color and opacity, and composited using front-to-back alpha blending. The shader is implemented in HLSL for the Universal Render Pipeline (URP) and supports single-pass instanced stereo rendering for VR through Unity's UNITY_SETUP_INSTANCE_ID and UNITY_INITIALIZE_VERTEX_OUTPUT_STEREO macros.

Key features of the ray marching implementation:

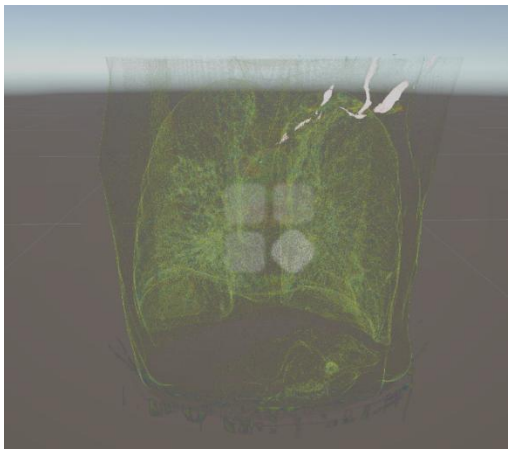
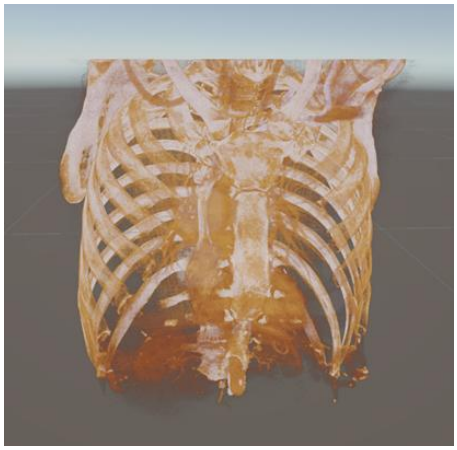
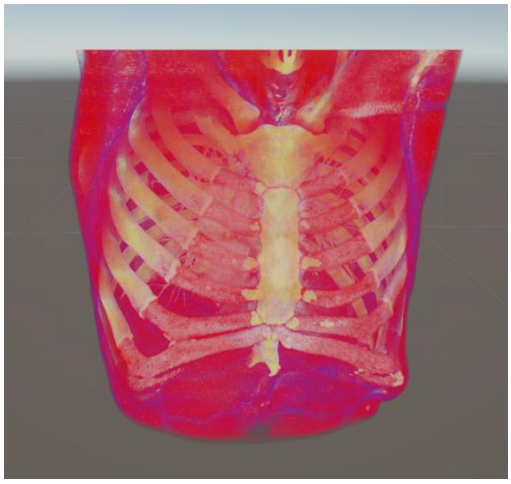
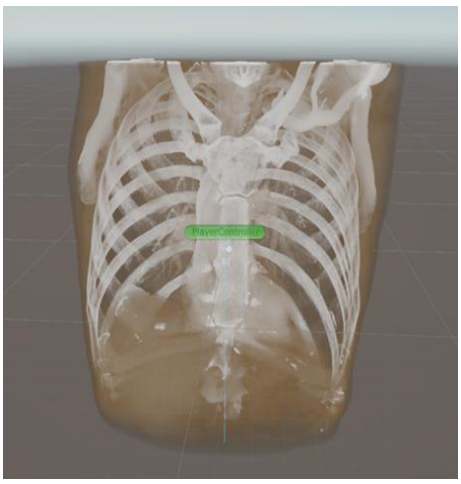
- **Configurable sampling rate:** The maximum step count (MAX_NUM_STEPS = 512) is multiplied by a runtime-adjustable _SamplingRateMultiplier property, allowing the step count to be tuned per platform. For Quest 3, this is set to 0.5 (256 effective steps) by the automation pipeline.

- **Opacity correction:** When the sampling rate changes, per-sample opacity is corrected by the formula $\alpha' = 1 - (1 - \alpha)^{(1/multiplier)}$ to maintain visually consistent density regardless of step count.
- **Early ray termination:** Controlled by the RAY_TERMINATE_ON shader keyword. When accumulated opacity exceeds the threshold (0.996), the ray loop terminates, avoiding unnecessary traversal of empty space behind opaque structures.
- **Jitter-based anti-aliasing:** A 512×512 noise texture offsets each ray's start position by a small random amount to suppress wood-grain banding artifacts inherent to uniform sampling.
- **Optional compile-time features:** Phong lighting with ambient/diffuse/specular (LIGHTING_ON), shadow volumes (SHADOWS_ON), tricubic interpolation (CUBIC_INTERPOLATION_ON), 2D transfer functions (TF2D_ON), cross-section plane culling (CROSS_SECTION_ON), depth buffer write (DEPTHWRITE_ON), and secondary volume overlay (MULTIVOLUME_OVERLAY/ISOLATE).

24.2 Transfer Functions

Three custom transfer functions were implemented based on 3D Slicer's preset library (presets.xml), each mapping Hounsfield Unit (HU) ranges to specific color and opacity curves. All three normalize raw HU values to the [0, 1] range using the dataset's actual minimum and maximum data values:

- **CT-Lung:** Targets lung parenchyma in the –600 to –399 HU range. The color palette transitions from blue (low density air) through green and yellow to red (higher density tissue). Maximum opacity is 0.15, producing a semi-transparent visualization of lung structures.
- **CT-Cardiac3:** Covers the full –3024 to 3071 HU range with cardiac-specific opacity and color curves optimized for visualizing heart chambers, vessels, and surrounding tissue.
- **CT-ChestContrastEnhanced:** Designed for contrast-enhanced chest CT scans, providing distinct visualization of vascular structures with enhanced vessel-to-tissue contrast.

24.3 Vertex Color Shader

A minimal custom shader (Medical/VertexColor) was written for segmentation mesh rendering. It uses single-pass vertex and fragment stages with back-face culling (Cull Back), a hardcoded directional light for basic shading via dot(worldNormal, lightDir), and per-vertex color output from the Marching Cubes mesh generation. Shadow casting is disabled to reduce GPU overhead.

25. Segmentation Mesh Generation

Although the primary output is volume rendering, a complete segmentation-to-mesh pipeline was developed for potential tumor and organ surface visualization. This pipeline is configurable via the BUILD_SEG_MESH parameter in the batch script (disabled by default).

25.1 Marching Cubes Algorithm

The Marching Cubes algorithm extracts an isosurface mesh from the blurred segmentation volume. Three execution modes are provided:

1. **Synchronous (Generate):** Processes the entire volume in a single call, suitable for small datasets or editor-time use.
2. **Chunked coroutine (GenerateChunked):** Yields control back to the Unity frame loop every N Z-slices (configurable via yieldEveryZSlices), preventing frame stalls during runtime generation.
3. **Jobs + Burst parallel (GenerateJobsAsync):** Each Z-slice is processed as an independent IJobParallelFor work unit compiled with [BurstCompile]. Per-slice triangle data is written to a NativeStream and read back after all jobs complete. This mode is the default for Quest builds.

All three modes share the same core lookup tables: a 256-entry edge table and a 256×16 triangle table implementing the standard Marching Cubes case resolution. Edge vertex positions are computed via linear interpolation between cube corner positions based on their scalar values relative to the iso-level.

25.2 Volume Smoothing

Before mesh extraction, a separable 3D box blur is applied to the binary segmentation volume to convert the discrete {0, 1} label field into a smooth [0, 1] float field. This produces rounded isosurface geometry instead of voxel-staircase artifacts. The blur uses three sequential 1D passes along the X, Y, and Z axes, reducing computational complexity from $O(n^3 \cdot k^3)$ for a direct 3D kernel to $O(n^3 \cdot 3k)$ for three separable passes (where k is the kernel radius). The iso-level is automatically set to 50% of the maximum blurred value when autoIso is enabled.

25.3 Taubin Mesh Smoothing

Post-extraction mesh smoothing uses the Taubin algorithm ($\lambda = 0.5$, $\mu = -0.53$) instead of simple Laplacian smoothing to prevent mesh shrinkage. The implementation uses a compressed sparse row (CSR) adjacency structure to efficiently handle meshes with 6M+ vertices. The CSR format stores all neighbor indices in a single flat array with a prefix-sum offset table, avoiding per-vertex hash map allocations. Each iteration consists of a shrink pass ($\lambda = 0.5$) followed by an inflate pass ($\mu = -0.53$). A NaN safety check on the first 100 vertices guards against numerically degenerate cases.

25.4 Mesh Assembly

The SingleMeshBuilder component orchestrates the segmentation pipeline end-to-end. It reads a NIfTI segmentation file via the custom reader, identifies all unique non-zero labels, and for each label: extracts a binary volume, applies the box blur, runs Marching Cubes (Jobs+Burst with chunked fallback), and accumulates vertices, triangles, and per-vertex colors into a unified mesh using UInt32 index format. Each anatomical label receives a distinct color from a 30-entry palette. The final mesh is rendered with the Medical/VertexColor shader with shadow casting disabled.

26. VR Interaction Design

The VR interaction model leverages Meta XR Building Blocks, which are installed automatically by the automation pipeline using reflection-based invocation of the Meta SDK's internal AddToProject API. The following interaction components are configured:

- **Camera Rig:** An OVRCameraRig with center eye anchor provides proper stereoscopic rendering and positional tracking.
- **Passthrough:** Mixed reality passthrough is enabled, allowing the user to see the real environment behind and around the CT volume, providing spatial context during examination.
- **Hand Tracking:** Real hand mesh visualization with hand grab interaction enables natural manipulation of the volume object, users can grab, rotate, and reposition the CT data using their hands.
- **Transfer Function Toggle:** The B button on the Quest 3 right controller toggles the CT-Lung transfer function overlay on or off at runtime. In Editor mode, the keyboard B key provides the same functionality.
- **Cross-Section Plane:** An interactive slicing plane (CT_Slicer) is spawned on the volume object, enabling cross-sectional anatomical exploration. The plane can be repositioned to reveal internal structures at any arbitrary orientation.

27. Performance Optimization for Quest 3

The Meta Quest 3 requires a sustained 72 Hz frame rate (approximately 11 ms per-frame GPU budget) for a comfortable VR experience. Volume rendering via ray marching is inherently GPU-intensive: at the Quest 3's native resolution of approximately 1832×1920 per eye, rendering both eyes with 512 samples per ray would require on the order of 3.5 billion texture samples per frame. A multi-layered optimization strategy was implemented to bring the frame time within budget without degrading diagnostic image quality.

27.1 GPU Optimizations

Technique	Implementation	Effect
Sampling Rate Reduction	_SamplingRateMultiplier = 0.5 (configurable via .bat)	512 → 256 steps/ray; ~50% GPU fragment reduction
Fixed Foveated Rendering	OVRManager FFR level HighTop with dynamic mode	~30–40% fragment shading savings in periphery
Shadow Removal	MainLightShadowsSupported = false in URP asset	Eliminates entire shadow map render pass
MSAA Reduction	MSAA 4x → 2x in Mobile_RPAsset	Halves multisample buffer memory and cost
Feature Stripping	Reflection probes, light layers, lens flare disabled	Removes unused per-frame render passes
Early Ray Termination	RAY_TERMINATE_ON keyword; break at $\alpha > 0.996$	Skips empty-space traversal behind opaque regions

27.2 CPU and System Optimizations

The QuestPerformanceSetup component manages system-level performance settings. It locks CPU and GPU clock levels to their maximum values (cpuLevel = 4, gpuLevel = 5) via the OVR API to prevent thermal-throttle-induced downclocking during sustained rendering. The display frequency is set to 72 Hz rather than 90 or 120 Hz to maximize the per-frame GPU time budget. Extra latency mode is enabled, adding one frame of pipeline latency in exchange for more consistent frame

delivery. The component automatically detects the headset model (Quest 2 vs Quest 3) via OVRPlugin.systemHeadsetType and applies hardware-appropriate presets. All settings can be overridden at runtime through a JSON configuration file (StreamingAssets/quest_performance.json) without requiring recompilation.

27.3 QuestVolumeOptimizer Integration

The QuestVolumeOptimizer component is the bridge between the automation pipeline and the volume rendering shader's sampling rate. The batch script defines a SAMPLING_RATE parameter (default: 0.5), which is passed to Unity as the -samplingRate command-line argument. AutoMedicalImport.cs reads this value, instantiates the QuestVolumeOptimizer component on the CT_Volume GameObject, and sets its samplingRate field before saving the scene. At runtime, the component's Start() method calls VolumeRenderedObject.SetSamplingRateMultiplier(), which updates the shader's _SamplingRateMultiplier uniform. The Inspector exposes a [Range(0.2, 1.0)] slider, enabling real-time visual comparison during development.

28. Render Pipeline Configuration

Two URP quality tiers are configured in the project. The Mobile tier targets Android/Quest builds, while the PC tier is used for desktop development and profiling.

Setting	Mobile (Quest)	PC (Editor)
Render Scale	0.8 (80%)	1.0 (100%)
MSAA	4x (target: 2x)	Disabled
HDR	Disabled	Disabled
Shadows	Enabled (target: Disabled)	Enabled
SRP Batcher	Enabled	Enabled
Post-Processing	Bloom (0.25) + Vignette (0.2)	Same
Stereo Rendering	Single-Pass Instanced (Multiview)	N/A

29. Project File Structure

File	Purpose
Run_Unity_Medical_Auto.bat	Entry point: all parameters, launches Unity
Assets/Editor/AutoMedicalImport.cs	Editor-time automation pipeline orchestrator
Assets/Scripts/NIfTIReader.cs	Custom NIfTI-1 parser for segmentation volumes
Assets/Scripts/MarchingCubesEngine.cs	Marching Cubes: sync, chunked, and Jobs+Burst
Assets/Scripts/MeshSmoother.cs	Taubin smoothing with CSR adjacency
Assets/Scripts/SingleMeshBuilder.cs	Segmentation mesh pipeline orchestrator
Assets/Scripts/CTLungTransferFunction.cs	3D Slicer CT-Lung preset implementation
Assets/Scripts/CTCardiac3TransferFunction.cs	3D Slicer CT-Cardiac3 preset implementation
Assets/Scripts/CtLungToggleRuntime.cs	Runtime B-button TF toggle controller
Assets/Scripts/QuestPerformanceSetup.cs	FFR, clock levels, display freq, headset detection
Assets/Scripts/QuestVolumeOptimizer.cs	Sampling rate optimization for volume rendering
Assets/Shaders/VertexColor.shader	Vertex color shader for segmentation meshes

Assets/Settings/Mobile_RPAsset.asset	URP pipeline asset for Quest builds

30. MedVisVR Application: Desktop Platform Overview

MedVisVR also has a desktop medical imaging application built with PyQt5 that provides an integrated environment for the segmentation and clinical review of three-dimensional volumetric image data in NIfTI format. The App's target audiences are Doctors and Medical students. The platform is designed to support fast clinical-style review workflows, enabling practitioners and researchers to load scan data, execute deep-learning-based segmentation pipelines, inspect output masks on a slice-by-slice basis, and export results for use in external tools such as 3D Slicer. The application is deployable on a standard clinical workstation without requiring internet connectivity, making it suitable for offline or air-gapped environments where patient data privacy and network isolation are priorities.

The system integrates two segmentation pathways: a brain MRI pipeline using nnU-Net v2 with multi-modal input (T1, T1ce, T2, FLAIR), and a lung CT pipeline using nnU-Net v1 (Task006_Lung). An embedded Assistant Web interface provides a locally hosted AI assistant backed by a FastAPI and Uvicorn server with llama.cpp inference, integrated directly into the application via a PyQt WebEngine widget. Patient data management is handled through a dedicated tab that lists segmentation outputs and allows users to open or reveal results in the file system.

The following table summarizes the key technical capabilities of the MedVisVR platform across its principal functional modules.

Component	Implementation
Brain MRI Segmentation	nnU-Net v2, multi-modal (T1, T1ce, T2, FLAIR), BraTS-trained weights, multi-class output (enhancing tumour, tumour core, whole tumour)
Lung CT Segmentation	nnU-Net v1 (Task006_Lung, nnunet_akciger_modeli/)
AI Assistant	Local FastAPI + uvicorn backend; llama.cpp GGUF inference; embedded via QWebEngineView at http://127.0.0.1:8000
3D Slicer Integration	One-click volume and mask handoff via configurable executable path; path persisted in .medvisvr_slicer.json
Patient Data Management	Timestamped NIfTI mask outputs under models/segmentation_outputs/; listed and accessible from Patient Data tab

Table 1. Summary of MedVisVR core components and their implementations.

31. MedVisVR GUI Architecture

MedVisVR is implemented as a PyQt5 desktop application. The graphical interface is divided into two principal layers: a login window that establishes user role context, and a main dashboard that exposes all functional capabilities through a tabbed layout. All UI components are contained within the ui/ directory of the project. The architectural separation between the UI layer and the inference services layer, implemented through worker classes in the services/ directory, follows the model-view separation principle, ensuring that computationally intensive inference operations do not block the GUI thread and that the interface remains responsive during long-running tasks.

31.1 Login Window

The login window provides a role selection interface with two options: Student and Doctor. The selected role is passed to the main dashboard and displayed in the application header, providing visual context throughout the session. This lightweight role context layer is designed to support extensibility: future iterations could use the role attribute to enforce feature-level access control, restrict certain diagnostic outputs to qualified clinical users, or customize the level of detail displayed in reports and confidence scores. In the current implementation both roles share access to the complete feature set, and the role selection functions primarily as a session label.

31.2 Main Dashboard and Tab Layout

The main dashboard is the central operational environment. It is structured around a persistent toolbar and a multi-tab content area. The toolbar exposes controls for organ and model selection, compute device switching between CPU and GPU, NIfTI file loading, segmentation execution, result saving, and 3D Slicer handoff. These controls are designed to remain accessible at all times regardless of the active tab, ensuring that the user can initiate or modify the core workflow from any view. The following table describes the four tabs comprising the application.

Tab	Function and Content
Segmentation	Primary working area for loading NIfTI volumes, configuring and running segmentation models, and visualizing the resulting mask overlaid on axial slices. Contains the slice viewer widget and all segmentation controls.
Assistant Web	Embedded QWebEngineView rendering the local assistant frontend served by the FastAPI backend at http://127.0.0.1:8000. Allows the user to query the AI assistant in natural language without leaving the application.
Patient Data	Lists all saved segmentation outputs from models/segmentation_outputs/ with filename, organ type, and creation timestamp. Supports open actions to reload a mask in the slice viewer and reveal-in-explorer actions for direct file system access.

Table 2. MedVisVR main dashboard tab descriptions.

31.3 Slice Viewer

The slice viewer is embedded within the Segmentation tab and constitutes the primary visual feedback component of the application. It renders NIfTI volumes on a slice-by-slice basis along the axial plane, overlaying the predicted segmentation mask using a colour-coded transparency scheme when a mask is available. Users scroll through slices interactively, enabling rapid visual inspection of segmentation quality across the full volume depth without requiring a separate external viewer. The slice viewer reads voxel data directly from the loaded NIfTI file and supports dynamic switching between the raw volume view and the mask overlay view, allowing users to compare pre- and post-segmentation representations of the same anatomical region.

32. MedVisVR Segmentation Pipelines

Segmentation is the core technical function of MedVisVR. The application supports two anatomical targets, brain and lung, through separate, specialised model pipelines whose design choices reflect the distinct imaging modalities, input requirements, and clinical conventions associated with each target. Pipeline selection is driven by the organ choice in the toolbar, and model management, input preprocessing, and inference orchestration are handled by the services/ directory, which implements a

consistent worker-based interface across all segmentation backends. This design ensures that the GUI layer remains decoupled from model-specific implementation details and that new pipelines can be introduced by adding a new worker class without modifying the interface layer.

32.1 Brain MRI Segmentation

Brain segmentation uses nnU-Net v2 with four co-registered MRI modalities as input: T1, T1ce, T2, and FLAIR. These are stacked into a multi-channel tensor and passed to the nnU-Net v2 inference engine, which produces a multi-class mask delineating enhancing tumour, non-enhancing tumour core, and peritumoral oedema in line with the BraTS challenge ontology. Each modality contributes complementary tissue contrast, reducing ambiguity between adjacent structures and improving delineation of the non-enhancing component. Output masks are saved to models/segmentation_outputs/ with a timestamp suffix.

32.2 Lung CT Segmentation

Lung CT segmentation uses nnU-Net v1 (Task006_Lung) with pretrained weights in nnunet_akciger_modeli/, trained on the Medical Segmentation Decathlon Task006_Lung dataset. The model accepts a single-channel NIfTI CT input and writes output masks under the timestamped convention in models/segmentation_outputs/. Its performance on that dataset, macro-averaged Dice of 0.8888 across 63 cases, is documented in Sections 17 to 21.

32.3 Output Handling and Traceability

All segmentation masks are written to models/segmentation_outputs/ with filenames encoding organ type and timestamp (e.g., brain_seg_20250101_120000.nii.gz). The original NIfTI header metadata, affine transformation and voxel spacing, is preserved in the output, ensuring spatial correspondence with the source volume in any downstream tool. The Patient Data tab enumerates this directory, displaying organ type and creation time for each entry, and provides open and reveal-in-explorer actions for further processing or archiving.

33. MedVisVR AI Assistant

33.1 Overview

The AI assistant module is a case-based clinical decision-support component embedded in MedVisVR via a QWebEngineView widget. The backend is hosted locally at http://127.0.0.1:8000 using FastAPI and Uvicorn, with inference served by llama.cpp from GGUF-format language model weights stored in the models/ directory. The assistant operates entirely offline, keeping all patient data and inference activity within the local machine. It is designed not as a general-purpose chatbot but as a structured pipeline that anchors every response to the quantitative outputs of the segmentation layer.

33.2 Planned Algorithms and Models

The assistant pipeline is composed of four sequential algorithmic stages. The question classification stage applies a lightweight rule-based and keyword-driven classifier to the incoming natural language question, assigning it to one of several predefined question types such as volumetric, localisation, multiplicity, enhancement, or high-risk. This classification determines the retrieval and generation strategy for the rest of the pipeline.

The structured retrieval stage maps the classified question type to the relevant fields of the case context object, which is populated from the segmentation output and includes tumour metrics, lesion list, atlas descriptors, and run metadata. Retrieval is deterministic and field-indexed: no vector search or embedding model is used at this stage. Each retrieved field is tagged with an evidence identifier that is carried forward into the response.

The answer generation stage operates in two modes depending on question type. For quantitative questions, volume, diameter, component count, the answer is assembled deterministically from retrieved structured fields with no language model involvement. For descriptive or interpretive

questions, a constrained prompt is constructed from the retrieved fields and passed to the local GGUF language model via llama.cpp. The prompt enforces case-grounding by injecting only the retrieved evidence fields and explicitly prohibiting extrapolation beyond the case data. High-risk question types, including diagnosis, treatment recommendation, prognosis, and drug dosing, bypass generation entirely and receive a structured refusal response produced by the deterministic layer.
The safety validation and scoring stage post-processes the generated answer before it is returned to the interface. A rule-based safety checker scans the answer text for disallowed claim patterns, escalatory language, and unsupported assertions. Where violations are found, the system applies soft repair by rewriting or removing the offending passage rather than discarding the response entirely. A confidence score is then assigned based on the number of resolved evidence fields, question type risk level, and whether LLM generation was involved. A safety note is appended to every response regardless of confidence level.

33.3 Report Generation Pipeline

The report generation pipeline produces a structured clinical draft report from the active case context. It is triggered explicitly by the user and operates in Clinical Safe Mode only. The pipeline constructs a compact report descriptor from the case context, covering tumour metrics, lesion-level summaries, atlas localization, and segmentation quality indicators, and uses this descriptor as the sole input to the language model, preventing the model from drawing on general knowledge beyond the current case.
The report is structured under six fixed sections: Clinical Context, Findings, Impression, Limitations, Recommendations, and Disclaimer. The Clinical Context, Limitations, and Disclaimer sections are produced deterministically from fixed templates and structured fields. The Findings section is assembled deterministically with optional LLM fluency applied to narrative transitions only. The Impression and Recommendations sections are generated by the language model under a constrained prompt that prohibits diagnostic claims, dosing guidance, and treatment decisions. A post-generation safety validation pass is applied to both LLM-assisted sections before the report is presented to the user.

33.4 Pipeline Outputs and Quality Metrics

Each assistant session produces a set of structured output artifacts written to the run output directory: findings_structured.json containing the structured case findings, tumor_metrics.json with quantitative tumour measurements, report_draft.md with the generated report, qa_results.jsonl with all question-answer pairs including evidence IDs and confidence scores, evidence.json mapping evidence identifiers to their source case fields, quality_report.json with session-level quality metrics, and run_meta.json storing model version, prompt policy, and run fingerprint for reproducibility.
The quality_report.json file records four session-level metrics computed across all QA responses in the session: the insufficient evidence rate (proportion of questions that received a structured refusal due to absent case-level support), the evidence coverage rate (proportion of answers with at least one resolved evidence identifier), the average confidence score, and the confidence band distribution showing the proportion of answers falling into high, medium, low, and insufficient tiers. These metrics are intended for audit and reproducibility review rather than real-time answer gating.

34. File Handling and NIfTI I/O

MedVisVR uses NIfTI as its primary input and output format throughout the segmentation workflow, consistent with the input conventions of nnU-Net and TotalSegmentator. For brain MRI, four NIfTI files (T1, T1ce, T2, FLAIR) are required; for lung CT, a single volume suffices. Loaded volumes are validated for dimensionality and voxel spacing before tensor conversion. Predicted masks are written back as NIfTI files preserving the source header metadata, so outputs are spatially registered to the input and compatible with any NIfTI-capable tool without additional steps. DICOM inputs are not natively supported and must be converted prior to use, for example using dcm2niix.

35. Compute Device Selection and Performance Considerations

MedVisVR supports both CPU and GPU execution for all segmentation and diagnosis inference operations. Device selection is exposed in the main toolbar as a toggle control and is applied uniformly across all active inference workers for the duration of the session. The selection propagates through the services/ layer to each worker at instantiation time, determining whether the model is loaded onto system RAM and executed on the CPU or loaded into GPU VRAM and executed via CUDA. The performance implications of this choice vary significantly across the different pipeline types, as summarised in the following table.

Pipeline	GPU Recommendation	Notes
nnU-Net v2 (Brain MRI)	Strongly recommended	Multi-channel, high-resolution 3D input requires 8+ GB VRAM. CPU inference may exceed 30 minutes per case.
nnU-Net v1 (Lung CT)	Recommended	Single-channel 3D CT; GPU reduces inference from several minutes to under 60 seconds in typical cases.

No automatic hardware detection or dynamic fallback logic is currently implemented in the codebase; the user is responsible for selecting the appropriate device based on the available hardware. Systems without a CUDA-compatible GPU can operate in CPU mode, but should expect substantially longer inference times, particularly for the brain MRI segmentation pipeline. Future work could introduce runtime CUDA availability checks and automatic device selection with appropriate user notification, as noted in the future improvements section.

36. Error Handling and User Feedback

MedVisVR validates prerequisites before any inference operation, verifying that required NIfTI files are loaded, model weights are present, and a compute device is set. Missing inputs trigger descriptive dialog boxes specifying exactly which requirement was unmet. All inference runs in background threads via the services/ worker classes, keeping the GUI responsive and communicating progress through toolbar status messages. Runtime exceptions, including CUDA out-of-memory, corrupted NIfTI, and missing weights, are caught within the worker layer and surfaced as error dialogs with suggested remediation steps. If the 3D Slicer executable path is invalid, the application prompts the user to reconfigure it rather than failing silently.

37. MedVisVR Segmentation Data Flow

The end-to-end data flow through MedVisVR for the primary segmentation workflow proceeds through five sequential stages, from initial user input through to final visualisation and result disposition. The following table provides a structured summary of each stage, its inputs and outputs, and the software components responsible for each transformation. This is followed by a description of the parallel diagnosis workflow that operates independently of the segmentation pipeline.

Stage	Description	Key Components
1. User Input	User selects organ target (brain or lung) and compute device (CPU/GPU) from the toolbar. NIfTI file(s) are loaded via file	Toolbar controls, file dialog, ui/ layer

Stage	Description	Key Components
	dialog. For brain: 4 modalities required (T1, T1ce, T2, FLAIR). For lung: single CT volume.	
2. Preprocessing	Loaded NIfTI files are validated for dimensions, voxel spacing, and data type. Volumes are converted to device-appropriate tensors. Brain modalities are stacked into a multi-channel input tensor.	services/ worker, nibabel / SimpleITK
3. Model Inference	Preprocessed tensor is dispatched to the selected model worker. Weights are loaded from the designated directory. Sliding window inference and test-time augmentation are applied. Forward pass executes on CPU or CUDA device.	services/ worker, nnU-Net v2 / v1
4. Output Generation	Raw model logits or probability maps are post-processed into a binary or multi-class mask. The mask is converted to NIfTI format preserving source header metadata (affine, spacing). Output is written to models/segmentation_outputs/ with timestamp.	services/ worker, models/segmentation_outputs/
5. Visualisation and Disposition	Slice viewer renders the volume and mask overlay. User inspects axial slices. Disposition options: save mask to Patient Data, open volume and mask in 3D Slicer, or review saved outputs from Patient Data tab.	Slice viewer (ui/), 3D Slicer subprocess, Patient Data tab

Table 3. MedVisVR segmentation data flow across the five processing stages.

38. References

[1] Isensee F, Jaeger PF, Kohl SAA, Petersen J, Maier-Hein KH. nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation. Nature Methods. 2021;18(2):203-211. DOI: 10.1038/s41592-020-01008-z.

[2] Menze BH, Jakab A, Bauer S, et al. The Multimodal Brain Tumor Image Segmentation Benchmark (BRATS). IEEE Transactions on Medical Imaging. 2015;34(10):1993-2024. DOI: 10.1109/TMI.2014.2377694.

[3] Abidi SA, Hoch MJ, Hu R, et al. Using Brain Tumor MRI Structured Reporting to Quantify the Impact of Imaging on Brain Tumor Boards. Tomography. 2023;9(2):859-870. DOI: 10.3390/tomography9020070.

[4] Manas-Garcia A, Gonzalez-Valverde I, Camacho-Ramos E, et al. Radiological Structured Report Integrated with Quantitative Imaging Biomarkers and Qualitative Scoring Systems. Journal of Digital Imaging. 2022;35(3):396-407. DOI: 10.1007/s10278-022-00589-9.

[5] Valerio AG, Trufanova K, de Benedictis S, Vessig G, Castellano G. From segmentation to explanation: Generating textual reports from MRI with LLMs. Computer Methods and Programs in Biomedicine. 2025;270:108922. DOI: 10.1016/j.cmpb.2025.108922.

[6] Essien M, Cooper ME, Gore A, et al. Interrater Agreement of BT-RADS for Evaluation of Follow-up MRI in Patients with Treated Primary Brain Tumor. AJNR American Journal of Neuroradiology. 2024;45(9):1308-1315. DOI: 10.3174/ajnr.A8322.

[7] Lei J, Zhang X, Wu C, et al. AutoRG-Brain: Grounded Report Generation for Brain MRI. CoRR abs/2407.16684. 2024. DOI: 10.48550/arXiv.2407.16684.

[8] Simpson AL, Antonelli M, Bakas S, et al. A large annotated medical image dataset for the development and evaluation of segmentation algorithms. arXiv preprint arXiv:1902.09063. 2019.

[9] Medical Segmentation Decathlon. http://medicaldecathlon.com

[10] M. Lavik, “UnityVolumeRendering: A Unity library for volume rendering of medical datasets,” GitHub Repository, 2023. [Online]. Available: https://github.com/mlavik1/UnityVolumeRendering

[11] F. Huettl, P. Saalfeld, C. Hansen, B. Preim, A. Poplawski, W. Kneist, H. Lang, and T. Huber, "Using virtual 3D-models in surgical planning: workflow of an immersive virtual reality application in liver surgery," Langenbeck's Archives of Surgery, vol. 406, pp. 911–920, 2021.

[12] L. Lan, R. Madani, E. Engel, and J. J. Martin, "Immersive virtual reality for patient-specific preoperative planning: a systematic review," *Surgical Innovation*, vol. 30, no. 1, pp. 109–122, 2023.

[13] V. Chheang, V. Fischer, R. Bruggermann, B. Preim, and C. Hansen, "Volumetric medical data visualization for collaborative VR environments," in *EuroVR 2020*, Lecture Notes in Computer Science, vol. 12499, Springer, 2020, pp. 42–58

[14] Isensee F, Jaeger PF, Kohl SAA, Petersen J, Maier-Hein KH. nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation. Nature Methods. 2021;18(2):203-211. DOI: 10.1038/s41592-020-01008-z.

[15] Menze BH, Jakab A, Bauer S, et al. The Multimodal Brain Tumor Image Segmentation Benchmark (BRATS). IEEE Transactions on Medical Imaging. 2015;34(10):1993-2024. DOI: 10.1109/TMI.2014.2377694.

[16] Abidi SA, Hoch MJ, Hu R, et al. Using Brain Tumor MRI Structured Reporting to Quantify the Impact of Imaging on Brain Tumor Boards. Tomography. 2023;9(2):859-870. DOI: 10.3390/tomography9020070.

[17] Manas-Garcia A, Gonzalez-Valverde I, Camacho-Ramos E, et al. Radiological Structured Report Integrated with Quantitative Imaging Biomarkers and Qualitative Scoring Systems. Journal of Digital Imaging. 2022;35(3):396-407. DOI: 10.1007/s10278-022-00589-9.

[18] Valerio AG, Trufanova K, de Benedictis S, Vessig G, Castellano G. From segmentation to explanation: Generating textual reports from MRI with LLMs. Computer Methods and Programs in Biomedicine. 2025;270:108922. DOI: 10.1016/j.cmpb.2025.108922.

[19] Essien M, Cooper ME, Gore A, et al. Interrater Agreement of BT-RADS for Evaluation of Follow-up MRI in Patients with Treated Primary Brain Tumor. AJNR American Journal of Neuroradiology. 2024;45(9):1308-1315. DOI: 10.3174/ajnr.A8322.

[20] Lei J, Zhang X, Wu C, et al. AutoRG-Brain: Grounded Report Generation for Brain MRI. CoRR abs/2407.16684. 2024. DOI: 10.48550/arXiv.2407.16684.

[21] Simpson AL, Antonelli M, Bakas S, et al. A large annotated medical image dataset for the development and evaluation of segmentation algorithms. arXiv preprint. 2019. arXiv:1902.09063.

[22] Wasserthal J, Breit HC, Meyer MT, Pradella M, Hinck D, Sauter AW, Heye T, Boll D, Cyriac J, Yang S, Bach M, Segeroth M. TotalSegmentator: Robust Segmentation of 104 Anatomic Structures in CT Images. Radiology: Artificial Intelligence. 2023;5(5):e230024. DOI: 10.1148/ryai.230024.